



BugIt

by Taskivator

Manuale completo passo dopo passo

BugIt

Agente QA per l'apertura di ticket di bug in VS Code

Configurazione iniziale e uso quotidiano: una guida dettagliata, clic dopo clic, scritta per chi non ha mai fatto nulla del genere. Non serve alcuna competenza tecnica. La segua in ordine e oggi stesso aprirà ticket di bug puliti.

Funziona in: VS Code su Windows (principale), macOS & Linux

Avviso sulla traduzione.

Questo documento è stato tradotto automaticamente e non è stato revisionato da madrelingua. Fa fede la versione inglese: in caso di difformità prevale il testo inglese. Per la formulazione più accurata e aggiornata, faccia riferimento al documento in inglese.

COSA TROVERAI

Segua la **Parte 1** una sola volta, in ordine. Tutto il resto può consultarlo quando le serve. Le sezioni **Risoluzione dei problemi** e **FAQ** verso la fine rispondono alle domande più comuni, quindi le controlli prima di scrivere al supporto.

Parte 1 · Configurazione (da fare una sola volta)

Passaggio 0: Si procuri GitHub Copilot

Passaggio 1: Installi VS Code

Passaggio 2: Estragga il download di Buglt

Passaggio 3: Apra Buglt in VS Code

Passaggio 4: Selezioni l'agente Buglt in chat

Passaggio 5: Attivi la sua licenza

Passaggio 6: Accetti i termini (una volta)

Passaggio 7: Esegua «Begin setup»

Passaggio 8: Colleghi il suo tracker

Passaggio 9: Salvi i suoi accessi

Passaggio 10: Confermi di essere pronto

Parte 2 · Uso quotidiano

Parte 3 · Lo personalizzi

Parte 4 · Aggiornamenti, backup e sicurezza

Parte 5 · Risoluzione dei problemi e FAQ

Parte 6 · Licenza e assistenza

La regola che la protegge sempre

Buglt **non** apre mai nulla senza prima mostrarle un'anteprima e ottenere la sua conferma digitata: per le azioni irreversibili digiti **FILE IT** ; un semplice «sì» non basta. Ha sempre il controllo. Vuole fare pratica a rischio zero? Digiti **dry run** e scrive l'intero report **senza** aprirlo.

Configurazione iniziale vs. uso quotidiano

Segua tutta la **Parte 1** una sola volta. Dopodiché, aprire Buglt è rapido, e gli aggiornamenti sono un solo comando (**Update**). Veda la Parte 4.

PARTE 1 · CONFIGURAZIONE: DA FARE UNA SOLA VOLTA

Circa 15 minuti. Segua i passaggi in ordine; ognuno si basa sul precedente. Non salti avanti.

Si procuri GitHub Copilot

Questa guida segue il percorso con GitHub Copilot

BugIt è la "guida" e l'AI è il "motore" (è quella che scrive davvero i report). Questi passaggi la configurano con GitHub Copilot in VS Code, il percorso più rapido se parte da zero. BugIt funziona anche con l'estensione Claude, con qualsiasi altro assistente in grado di eseguire un comando e da solo con la sua chiave AI. Se ne usa già uno, passi al Passo 2 e apra la cartella di BugIt in quello strumento anziché in VS Code.

Cos'è: GitHub Copilot è un assistente AI di GitHub (di proprietà di Microsoft). Ha un piano gratuito per un uso occasionale e uno a pagamento per un uso intenso. Lo installerà al Passaggio 1 e accederà.

1. Le serve un **account GitHub**. Se non ne ha uno, ne crei uno gratuito su github.com/signup.
2. Il piano gratuito di Copilot basta per provare BugIt. Se apre bug tutto il giorno, vale la pena passare al piano a pagamento "Copilot Pro", e può passare in qualsiasi momento su github.com/features/copilot.
3. Attiverà davvero Copilot dentro VS Code al Passaggio 1, quindi per ora non serve fare altro qui.

Installi VS Code

VS Code è un'app gratuita di Microsoft. È la "finestra" in cui vive BugIt.

1. Apra il browser e vada su code.visualstudio.com.
2. Clicchi il grande pulsante blu **Download** (rileva automaticamente il suo sistema).
3. Apra il file scaricato e lo installi: **accetti tutte le opzioni predefinite**, basta continuare a cliccare Next/Install.
4. Apra VS Code una volta installato.

Attivi GitHub Copilot dentro VS Code

1. Sulla sinistra di VS Code, clicchi l'icona **Estensioni** (sembra quattro piccoli quadrati).
2. Nella casella di ricerca, digiti **GitHub Copilot**.
3. Clicchi **Install** su "GitHub Copilot" (e, se proposto, anche su "GitHub Copilot Chat").
4. Un messaggio le chiederà di **Sign in to GitHub**. Lo clicchi e acceda con il suo account GitHub nella finestra del browser che si apre.

✔ **Saprà che ha funzionato quando:** una piccola icona di Copilot appare in basso in VS Code, e si apre il pannello Chat (l'icona a forma di fumetto a sinistra, oppure preme **Ctrl+Alt+I**).

2

Estragga il download di Buglt

Il suo acquisto include un file .zip (per esempio `bugit-qa-agent.zip`). Deve estrarlo; eseguirlo ancora compresso non funziona.

1. Trovi il file .zip (di solito nella cartella **Download**).
2. **Clic destro** su di esso → scelga **Extract All...** (Windows) oppure faccia doppio clic (Mac).
3. Scelga una posizione semplice come la cartella **Documenti**, poi clicchi Extract.
4. Avrà ora una cartella **Buglt** con tanti file all'interno. Ricordi dove si trova.

Non metterla in OneDrive o in una cartella sincronizzata

Le cartelle sincronizzate nel cloud possono causare conflitti tra i file. La cartella Documenti o il Desktop vanno benissimo. Inoltre non rinomini i file interni; lasci la cartella così com'è.

3

Apra Buglt in VS Code

1. In VS Code, clicchi **File** → **Open Folder** (menu in alto a sinistra).
2. Vada dove ha estratto Buglt (es. Documenti).
3. Clicchi una volta sulla cartella **Buglt** per selezionarla, poi clicchi **Select Folder**.
4. Se VS Code le chiede se si fida degli autori dei file in questa cartella, clicchi **Yes, I trust the authors**.

✓ **Saprà che ha funzionato quando:** vede i file di Buglt (cartelle come `tools`, `.github`, e file come `README.md`) elencati sul lato sinistro di VS Code.

4

Selezioni l'agente Buglt in chat

1. Apra il pannello **Chat** di Copilot: clicchi l'icona a forma di fumetto a sinistra, oppure prema `Ctrl+Alt+I` (Windows) / `Cmd+Alt+I` (Mac).
2. In cima alla casella della chat c'è un piccolo **menu a tendina della modalità** (potrebbe indicare "Ask" o "Agent").
3. Lo clicchi e scelga **Buglt QA Agent** dall'elenco.

Non vede "Buglt QA Agent" nell'elenco?

Si assicuri di aver aperto la cartella Buglt (Passaggio 3), non un'altra cartella; l'agente si carica da lì. Chiuda e riapra la cartella se necessario.

5

Attivi la sua licenza

Le basta un account Buglt; non c'è alcun codice di licenza da copiare o incollare. L'attivazione avviene nel suo browser.

1. Nella casella della chat, digiti **hi** e prema Invio.
2. Digiti **Activate**. Buglt apre il Buglt Portal nel suo browser. Acceda al suo account, scelga il suo diritto Solo o Team e approvi questo dispositivo.
3. Torni in VS Code e Buglt completa automaticamente l'autorizzazione; la sua password resta nel browser e non viene mai inserita in VS Code.

✓ **Saprà che ha funzionato quando:** Buglt conferma l'attivazione e indica l'account con cui ha effettuato l'accesso.

Tenga privato il suo account

Il suo account e il suo diritto sono legati a lei. Non li condivida, pubblichi o rivenda; un accesso condiviso può essere revocato. I membri di un Team accedono ciascuno con il proprio account: non esistono credenziali condivise di alcun tipo.

6

Accetti i termini (una volta)

La prima volta, Buglt mostra un breve riepilogo e le chiede di accettarlo prima di fare qualsiasi altra cosa.

1. Legga il breve riepilogo che mostra (riveda ogni ticket prima che venga aperto; la sicurezza del suo progetto è una sua responsabilità; le misure di sicurezza sono un aiuto, non una garanzia, e poiché vengono seguite dal modello AI che le risponde in Copilot, un modello più leggero/gratuito potrebbe occasionalmente saltare un passaggio o richiedere che un comando venga ripetuto, cosa che un modello più potente non farebbe).
2. Scelga **I agree** fra le due opzioni mostrate da Buglt.

✓ **Saprà che ha funzionato quando:** Buglt conferma e passa alla configurazione. Glielo chiederà solo questa volta; se lo ricorda.

7

Esegua «Begin setup»

Ora Buglt si configura da solo intervistandola in linguaggio semplice. Non modifica mai un file di impostazioni a mano.

LEI DIGITA

Begin setup

Farà brevi domande e attiverà solo ciò che sceglie. La più importante riguarda il suo tracker.

8

Colleghi il suo tracker

La configurazione ha registrato *quale* tracker lei usa. Questo passaggio è ciò che consente a BugIt di registrare davvero i ticket: crei un token API nel suo account e lo salvi su questo computer con un solo comando. BugIt registra i ticket direttamente tramite l'API REST del suo tracker con quel token, quindi non deve restare in esecuzione nulla e la registrazione non usa mai un server MCP.

8a · Crei un token nel suo account

IL SUO TRACKER	DOVE CREARLA
Jira Cloud	id.atlassian.com/manage-profile/security/api-tokens
Azure DevOps	dev.azure.com/YOUR-ORG/_usersSettings/tokens
GitHub Issues	github.com/settings/personal-access-tokens/new
GitLab Issues	gitlab.com/-/user_settings/personal_access_tokens
Bugzilla	nel suo Bugzilla: Preferences → API Keys
YouTrack	nel suo YouTrack: avatar → Profile → Account Security → Authentication
Linear	linear.app/settings/account/security
Shortcut	app.shortcut.com/settings/account/api-tokens
ClickUp	app.clickup.com/settings/apps
Asana	app.asana.com/0/my-apps
Trello	trello.com/apps/admin (una chiave API E un token che autorizza con essa)

Copi il suo token appena viene mostrato

La maggior parte dei servizi mostra un nuovo token una sola volta. Lo copi subito e lo conservi in un posto sicuro finché BugIt non lo salva per lei (Passaggio 9). Scelga la scadenza più lunga che il suo tracker offre, così non dovrà rifarlo a breve.

8b · Lo salvi su questo computer

Apra un terminale dentro VS Code ed esegua il comando del suo tracker, usando `ado`, `github`, `gitlab` e così via al posto di `jira`.

LEI ESEGUE (NEL TERMINALE)

```
python tools/connect.py jira
```

8c · Incolli il token nella richiesta mascherata

Il comando chiede il token in una richiesta locale mascherata: non compare mai sullo schermo e non raggiunge mai la chat, un ticket o `config.json`. Lo incolli lì e in nessun altro posto. BugIt accede, elenca i progetti che il token vede davvero, legge i tipi di ticket che lei può creare e **rifiuta di salvare una credenziale che non può registrare**. Se la configurazione si conclude, la registrazione funziona.

8d · Verifichi che sia collegato

Per averne conferma in qualsiasi momento, esegua la verifica nel terminale oppure lo chieda in chat:

LEI ESEGUE (NEL TERMINALE)

```
python tools/connect.py jira --check
```

LEI DIGITA

```
Check status
```

✔ **Saprà che ha funzionato quando:** BugIt elenca il suo tracker come collegato / in buono stato.

Non deve lasciare nulla in esecuzione

Il suo token API resta nell'archivio credenziali del suo sistema operativo, non in VS Code. Quando riapre VS Code non c'è quindi nulla da avviare: BugIt può registrare subito. Può verificarlo quando vuole con `python tools/connect.py jira --check`.

9

Salvi i suoi accessi

Quando salva il suo token (Passaggio 8c), le credenziali vengono salvate in modo sicuro nel vault integrato del suo computer (Windows Credential Manager / macOS Keychain / Linux Secret Service), mai in un file in chiaro e mai inviate da nessuna parte. Non deve fare altro: non dovrà più cercare il suo token a ogni riavvio.

Per vedere quali tracker hanno già una credenziale salvata, digiti:

LEI DIGITA

```
Check saved credentials
```

...e BugIt mostra **quali** tracker hanno una credenziale salvata, mai i valori delle credenziali, che non vengono mai mostrati né copiati.

10

Confermi di essere pronto

Chieda a BugIt di ricontrollare tutto prima di aprire ticket sul serio:

LEI DIGITA

Check readiness

Elenca tutto ciò che manca ancora (di solito solo "colleghi il suo tracker" o "attivi"). Quando è tutto verde, vedrà "✅ Configurazione completata."

Configurazione fatta, per sempre

Non ripeterà la Parte 1. Da qui in poi, aprire BugIt significa: apra la cartella → digiti `hi`. Per cambiare una scelta in seguito, digiti semplicemente `Begin setup` e dica cosa cambiare.

Niente GitHub Copilot? C'è una procedura guidata da terminale

Se non può usare Copilot, apra il Terminale integrato (Terminal → New Terminal) ed esegua `python tools/setup_wizard.py`. Fa qualche domanda e scrive le sue impostazioni senza l'AI. Può anche eseguire BugIt in modalità standalone con la sua chiave AI; veda le FAQ.

PARTE 2 · USO QUOTIDIANO

Una volta configurato, lavora interamente dalla chat. Parli in modo naturale, oppure usi gli esempi esatti qui sotto. Digiti `help` in qualsiasi momento per l'elenco completo.

Scriva un report di bug

Il compito principale di BugIt. Descriva il bug in una frase semplice; scrivi il ticket completo, compila automaticamente versione e categoria, controlla i duplicati, le mostra un'anteprima e lo apre dopo la sua conferma digitata (digiti `FILE IT`; un semplice «sì» non basta).

LEI DIGITA

Write a bug report: l'app si chiude ogni volta che tocco Salva su iPhone

Prepara un titolo, i passaggi per riprodurlo, il risultato atteso e quello effettivo, la gravità e la piattaforma, e imposta sempre il campo di priorità/gravità del tracker stesso in base a questi dati, non un valore predefinito generico. Vuole delle modifiche prima di aprirlo? Lo dica e basta, per esempio "aumenta la gravità" oppure "aggiungi che è iniziato con l'ultima build."

Bug rapidi (apertura veloce)

Per le casistiche più comuni (crash, error, missing, layout, slow, stuck, data, visual, audio, blocker), il modulo rapido salta alcune domande e compila categoria e priorità per lei, e salta la ricerca dei duplicati per risparmiare il viaggio di andata e ritorno. Lo usi quando sa già che il problema è nuovo; l'anteprima glielo dice, e un report standard esegue comunque il controllo completo dei duplicati.

LEI DIGITA (ESEMPI)

Quick bug: crash all'avvio dopo l'aggiornamento

Quick bug: layout il pulsante si sovrappone al testo nella schermata delle impostazioni

Quick bug: blocker non riesco a proseguire oltre la fase di pagamento

Quick bug: slow la dashboard impiega 20 secondi a caricarsi

Report di bug per i crash

Se ha collegato uno strumento per i crash (come Sentry), Buglt fa tutto quanto sopra e in più abbina il suo report al crash e recupera lo stack trace per lei.

LEI DIGITA

Write a crash bug report: il gioco va in crash quando apro la mappa

Commenti di verifica e chiusura

Dopo aver verificato una correzione, Buglt scrive un commento ordinato da pubblicare sul ticket. Scrive (ed eventualmente pubblica) il commento; non cambia lo stato del ticket. È lei a chiudere/risolvere/riaprire il ticket nel suo tracker.

LEI DIGITA	COSA OTTIENE
Close #123	Chiede quale scenario (correzione confermata, chiuso su richiesta, comportamento previsto, riapertura...) poi scrive quel commento.
Write a verification comment for the build you tested on Windows	Un commento "correzione confermata" con i dettagli della build già compilati.
Write a reopen comment for the build you tested on Windows	Un commento "il problema persiste" (poi riapre lei stesso il ticket).

Traduzione

Traduce report o termini tra le lingue configurate, usando la formulazione esatta del suo team (dal suo glossario; veda la Parte 3).

LEI DIGITA (ESEMPI)

Translate to ja: il pulsante salva non fa nulla nella schermata del profilo

Translate this bug report to English: [incolla il testo]

Cerchi informazioni (specifiche e glossario)

Se ha collegato le specifiche o compilato il suo glossario, faccia domande a BugIt sul suo progetto.

LEI DIGITA (ESEMPI)

Cos'è [il suo termine]?

Guidami attraverso il flusso di checkout

Cosa c'è di nuovo nelle specifiche?

Rilevamento dei duplicati

Avviene automaticamente in un report standard, prima che venga scritto qualsiasi cosa: BugIt cerca nel suo tracker ticket corrispondenti (anche formulati in modo un po' diverso) e la avvisa, così non apre mai due volte lo stesso bug. Il modulo rapido salta la ricerca di proposito per risparmiare il viaggio di andata e ritorno, e lo dice nell'anteprima, così sa sempre quale dei due ha ottenuto.

Funzionalità avanzate (se le ha attivate)

DIGITI QUESTO	COSA FA
<code>Triage digest</code>	Standup istantaneo: bug aperti per area/gravità, i più vecchi segnalati.
<code>Export bug to file</code>	Salva il report come Markdown/CSV/JSON invece di aprirlo (ottimo prima che il suo tracker sia configurato).
<code>Undo last filing</code>	Mostra l'ultima scrittura nel log di controllo. BugIt non può eliminare né chiudere un ticket; dopo una nuova anteprima e <code>FILE IT</code> può rimuovere solo un commento o un allegato creati da lui.
(automatico)	Segnalazioni SLA sui bug urgenti · campi specifici per categoria.

L'elenco completo dei comandi

LEI DIGITA

`help`

...mostra ogni comando disponibile nell'agente, in qualsiasi momento.

Faccia pratica in sicurezza in qualsiasi momento

Digiti `dry run` e BugIt scrive l'intero report **senza** aprire nulla, perfetto per imparare o fare test.

PARTE 3 · LO PERSONALIZZI: AGGIUNGA LA CONOSCENZA DEL SUO PROGETTO

Appena installato, BugIt è un agente QA a foglio bianco. Ecco come insegnargli il suo progetto dopo l'acquisto. Nessuna programmazione, quasi solo parlargli in chat. Tutto ciò che aggiunge resta sul suo computer.

1 · Le parole del suo team (il glossario)

Questo rende accurate le traduzioni e l'individuazione della categoria per il suo progetto.

1. Nell'elenco dei file, apra `.github/glossary/terms.template.md`.
2. Compili la tabella con le parole del suo team: nomi delle funzionalità, nomi delle schermate, nomi degli elementi, abbreviazioni e (se usa due lingue) le loro traduzioni.
3. Salvi il file (`Ctrl+S`). Tutto qui: da questo momento Buglt usa esattamente questi termini.

Scorciatoia

Attivi l'"auto-popolamento del glossario" durante `Begin setup` e Buglt suggerisce termini dai suoi ticket esistenti, e lei approva ognuno.

2 · Il suo stile dei ticket (house style)

Vuole che i ticket rispecchino esattamente il formato del suo team: stile del titolo, etichette di gravità, campi obbligatori? Non lo descrive; lo **mostra**:

1. In chat, dica: "adattati allo stile del mio team".
2. Incolli **uno screenshot** di un ticket esistente dal suo tracker.
3. Buglt lo analizza (in locale, con eventuali dati personali rimossi prima) e salva il suo formato, poi lo segue in ogni ticket futuro.

3 · Le sue categorie, piattaforme e campi

Lo dica semplicemente a Buglt in linguaggio naturale e aggiorna le sue impostazioni per lei:

LEI DIGITA (ESEMPLI)

Le mie categorie sono Gameplay, UI, Audio e Networking
Facciamo test su Windows e Xbox

4 · I suoi strumenti personalizzati (connessioni personalizzate)

Usa uno strumento che non è nell'elenco integrato? Lo colleghi proprio come uno predefinito:

LEI DIGITA

Add a custom MCP server

Gli dia un nome breve e il suo indirizzo (deve iniziare con `https://`); Buglt lo collega e ne verifica lo stato di salute per lei.

5 · Lo corregga semplicemente man mano

Il modo più semplice di tutti: quando Buglt prepara un ticket, corregga qualsiasi cosa non le piaccia: un'etichetta, la formulazione, la gravità. Con l'opzione "impara dalle tue modifiche" attivata (consigliata), ricorda la sua preferenza e la applica la volta successiva. La conoscenza del suo progetto si accumula naturalmente, e non lascia mai la sua macchina.

Tutto ciò che aggiunge è suo e resta in locale

Il suo glossario, lo stile dei ticket, le categorie e le preferenze apprese vivono sul suo computer, vengono salvati in backup prima di ogni aggiornamento e non vengono mai inviati a nessuno.

6 · Condividi la sua configurazione con il team (licenza Team)

Una volta impostati glossario, stile dei ticket e scelta del tracker come preferisce, dia al resto del suo team esattamente la stessa configurazione con un solo comando, invece di ripetere i passaggi 1 - 3 su ogni macchina:

LEI ESEGUE (UNA VOLTA, DOPO CHE LA SUA CONFIGURAZIONE È COME LA VUOLE)

```
python tools/share.py export
```

Questo scrive `config.share.zip`: le sue scelte di tracker/crash/comunicazioni, il glossario e lo stile dei ticket, raggruppati insieme senza password o token all'interno. Lo invii al suo team.

OGNI COLLEGA ESEGUE

```
python tools/share.py import config.share.zip
```

Otengono immediatamente la sua terminologia esatta, il suo stile dei ticket e la configurazione del tracker, senza ridigitare le categorie e senza reincollare uno screenshot per lo stile. La loro licenza e le impostazioni del dispositivo restano invariate.

Se l'importazione cambia dove vengono inviati i dati

Se una configurazione condivisa aggiunge una nuova connessione personalizzata o un indirizzo di notifica, Buglt mostra esattamente cosa è cambiato prima che accada qualsiasi altra cosa; non viene applicato silenziosamente. Inoltre crea prima un'istantanea della configurazione attuale del collega, così `Restore my settings` annulla l'importazione se non è quello che si aspettava.

PARTE 4 · AGGIORNAMENTI, BACKUP E SICUREZZA

Ricevere gli aggiornamenti (un solo comando)

Quando è disponibile una nuova versione, Buglt glielo segnala una volta quando digita `hi`. Gli aggiornamenti del software sono inclusi finché la sua licenza Buglt è attiva. Per installarla:

LEI DIGITA

```
Update
```

1. Buglt esegue prima un backup aggiornato dei suoi file.
2. Scarica la nuova versione e verifica che sia autentica e non manomessa (è firmata crittograficamente, quindi un aggiornamento falso o alterato viene rifiutato).

3. La installa senza toccare le sue impostazioni, il glossario, lo stile dei ticket, la licenza o i token.
4. Le chiede di ricaricare VS Code (`Ctrl+Shift+P` → digiti "Reload Window" → Invio). Avvii una nuova chat ed è sulla nuova versione.

Mai eliminare cartelle

A differenza della prima installazione, gli aggiornamenti avvengono sul posto. Non elimina né ricarica mai nulla, e la sua configurazione viene sempre preservata e salvata in backup.

Cosa è sicuro modificare a mano e cosa no

Sicuro, e conservato a ogni aggiornamento: `config.json`, `.github/instructions/house-style.instructions.md` (veda la Parte 3; questo è il modo supportato per personalizzare il comportamento), `.github/glossary/terms.template.md`, `.github/instructions/learned.instructions.md` e `.vscode/mcp.json`.

Non sicuro: non li modifichi a mano

Il file dell'agente (`.github/agents/bugit-qa-agent.agent.md`), qualsiasi altro file sotto `.github/instructions/` e tutto ciò che si trova in `tools/` sono il prodotto stesso, non le sue impostazioni. Un aggiornamento li **sovrascrive silenziosamente**, e, a differenza dei file sopra, **non esiste un backup da cui ripristinarli**. BugIt controlla questo all'avvio e di nuovo subito prima di installare un aggiornamento, e la avvisa se trova una modifica, ma la regola più sicura è semplicemente non modificarli.

Backup e ripristino

LEI DIGITA	COSA FA
<code>Back up my settings</code>	Salva un'istantanea locale della sua configurazione, del glossario, dello stile dei ticket, degli endpoint MCP e dell'apprendimento; il suo diritto del dispositivo firmato e i valori delle credenziali sono esclusi.
<code>List backups</code>	Mostra ogni istantanea salvata con la sua data.
<code>Restore my settings</code>	Ripristina un'istantanea (e salva prima un'istantanea del suo stato attuale, così anche un ripristino è annullabile).

Automatico prima di ogni aggiornamento

BugIt esegue sempre un backup prima di installare una nuova versione, così un aggiornamento non può mai far perdere la sua configurazione. Se qualcosa non sembra giusto, `Restore my settings` lo sistema.

Sicurezza: cosa è protetto

✓ Nulla senza la sua conferma

Ogni ticket e commento viene mostrato in anteprima; le azioni irreversibili richiedono di digitare `FILE IT`, e un semplice «sì» non basta.

🧪 Dry-run = zero scritture

Dica `dry run` e il suo tracker non viene contattato affatto, letture comprese. Il dry-run sopprime ogni ticket, commento, modifica, allegato, rimozione e notifica esterni (i comandi locali che lei avvia di proposito, come backup o esportazione, possono ancora creare file locali) e non c'è nessuna ricerca, nessun controllo dei duplicati e nessuna verifica della connessione, perché ognuna di esse raggiunge la sua board e sblocca il token salvato.

`QA_AGENT_DRY_RUN=1` nel terminale è l'interruttore che i comandi inclusi applicano nel codice; dire «dry run» in chat chiede all'assistente di fermarsi, il che è utile ma non è la stessa garanzia.

🔍 Nessun segreto nei file

I token vivono nel vault del suo sistema operativo, mai in un file in chiaro, mai inviati a noi.

🔒 Funziona sulla sua macchina

Comunica solo con gli strumenti che collega e il suo modello AI.

I suoi dati e la privacy

L'unica cosa che Buglt invia a Taskivator sono i dati di licenza/aggiornamento: l'accesso al suo account Buglt (nel browser, sul Portal), un ID dispositivo con hash a senso unico, un identificativo di installazione casuale, l'etichetta del suo dispositivo (nome host) e il nome del suo sistema operativo, la versione di Buglt e il filtro di piano che sceglie, materiale crittografico di sfida a vita breve, e il diritto Solo o Team che approva per questo dispositivo. Non c'è alcun codice di licenza, e la sua password resta nel browser e non viene mai inserita in VS Code. I suoi report di bug, le specifiche, il glossario, le impostazioni e i token non lasciano mai la sua macchina. Nessuna telemetria, nessuna analisi, mai. I dettagli completi sono nel file `PRIVACY.md` nella sua cartella Buglt.

Rischi e avvertenze importanti: legga con attenzione

L'AI può sbagliare

Buglt scrive i report con un modello AI, che può interpretare male i dettagli o inventare qualcosa. Legga sempre l'anteprima prima di confermare. È lei ad approvare ciò che viene aperto.

Apri ticket sul suo tracker reale

Una volta approvato, viene creato un ticket vero. Sta facendo pratica? Usi `dry run` per scrivere il report senza aprirlo.

La sicurezza del suo progetto è sua

Come usa Buglt, e la sicurezza dei suoi progetti, dati e tracker, sono una sua responsabilità. Le misure di sicurezza riducono il rischio ma non sostituiscono la sua revisione.

Cosa non fa Buglt

Buglt registra in tutti e undici i tracker che nomina, ciascuno con una credenziale che lei crea nel proprio account e che Buglt verifica contro la destinazione scelta prima di salvarla. Ciò che arriva in un campo priorità ordinabile cambia da tracker a tracker, e GitHub, GitLab e Trello non ne hanno alcuno, quindi lì la gravità viene scritta nel testo del ticket. Confluence, Sentry e Notion sono di sola lettura; tutto il resto si collega tramite il suo server MCP compatibile, che configura lei con l'aiuto di Buglt. Usa la sua IA (Copilot o la sua chiave OpenAI/Anthropic), non un modello incluso; l'oscuramento è al meglio possibile; e funziona in VS Code, nell'estensione Claude e in un normale terminale. Una licenza = un dispositivo (Solo) oppure fino a 5 membri o postazioni (Team). Risultati e coerenza dipendono anche da quale modello di IA risponde: uno più potente segue i passaggi di sicurezza in modo più affidabile di uno molto leggero o gratuito.

PARTE 5 · RISOLUZIONE DEI PROBLEMI E FAQ

Quando qualcosa non va, lo descriva prima all'assistente in parole semplici. La maggior parte dei problemi di configurazione e di uso quotidiano si diagnostica direttamente nell'editor.

✦ Provi prima questo: chiedi all'AI di risolverlo

Ogni volta che qualcosa va storto (durante la configurazione o nell'uso quotidiano), la soluzione più rapida è quasi sempre **chiedere all'assistente direttamente in chat**. Descriva cosa è successo con parole semplici (per esempio, "la configurazione si è fermata a metà" oppure "il mio tracker non si connette") e gli chiedi di risolverlo. L'AI può diagnosticare e riparare la maggior parte dei problemi sul momento. Provi questa strada prima di scriverci un'email; risolve la grande maggioranza dei problemi in pochi secondi.

Applicare una correzione dell'AI: il pulsante "Keep"

Se l'assistente risolve qualcosa modificando un file, VS Code mostra una piccola barra Keep / Undo. Se ha chiesto lei la correzione e ne è soddisfatto, clicchi **Keep**, e la modifica viene applicata solo alla sua copia, sul suo computer. Clicchi **Undo** per scartarla. Nulla che tiene con Keep viene condiviso con nessun altro.

COSA VEDE	COSA FARE
"Attiva per iniziare"	Digiti Activate . Buglt apre il Buglt Portal nel suo browser. Acceda, scelga il suo diritto Solo o Team e approvi questo dispositivo. Non ha ancora un account? Ne crei uno sullo store.
Il browser non si è aperto, oppure l'attivazione non si è completata	Digiti di nuovo Activate per riaprire il Buglt Portal, poi completi l'accesso e l'approvazione di questo dispositivo. L'attivazione avviene interamente nel browser; non c'è nulla da incollare.
"Nessun tracker abilitato"	Digiti Begin setup e scelga il suo tracker.
La registrazione viene rifiutata	Esegua python tools/connect.py jira --check . Le dice chi è per il tracker, che cosa Buglt può creare, ed elenca in fixes il rimedio preciso.

Continua a chiederle di attivare	Digiti <code>Activate</code> e completi l'accesso nel browser. BugIt non mostra mai i valori delle credenziali salvate. (Prima volta? Veda il Passaggio 8c.)
Non apre un bug	Digiti <code>Check readiness</code> ; elenca esattamente cosa manca (di solito il token del tracker non è ancora stato salvato).
Qualcosa sembra rotto dopo un aggiornamento	Digiti <code>Restore my settings</code> per tornare indietro.
L'agente sembra "fuori copione"	Digiti <code>Read and follow all your given instructions</code> , oppure avvii una nuova chat. Questo può succedere più spesso con un modello AI leggero/gratuito; un modello più potente nel selettore di Copilot è più coerente.
Le risposte sembrano generiche / non specifiche per il progetto	Gli dia più contesto, oppure gli mostri un ticket esistente durante <code>Begin setup</code> così impara il suo stile. Riesegua <code>Begin setup</code> per affinarlo.
Vuole cambiare una scelta di configurazione	Digiti <code>Begin setup</code> e dica cosa cambiare (per esempio "aggiungi un tracker" o "ricomincia da capo") e riapre solo quella parte. Se la configurazione è stata semplicemente interrotta a metà, un semplice <code>Begin setup</code> riprende da dove si era fermata invece di ricominciare.

Controlli di salute integrati

`Check status` (i miei strumenti sono collegati?) · `Check readiness` (cosa manca prima di poter aprire ticket?). Per un controllo più approfondito, apra Terminal → New Terminal ed esegua `python tools/selftest.py` .

Domande frequenti

Devo sapere programmare?

No. Se sa installare un'app e scrivere una frase, può usare BugIt. Si occupa lui delle parti tecniche.

Aprirà bug senza che io lo sappia?

Mai. Ogni ticket e commento viene mostrato in anteprima e le azioni irreversibili richiedono di digitare `FILE IT` , e un semplice «sì» non basta. Usi `dry run` per fare pratica senza alcuna scrittura.

Devo avviare un server ogni volta?

No. La registrazione usa il token API che ha salvato con `python tools/connect.py` , quindi non c'è nulla da avviare. Un server MCP serve solo per le integrazioni facoltative di sola lettura, come Confluence, Sentry e Notion.

Posso usarlo su due computer?

Un dispositivo alla volta con Solo (un massimo di 5 membri con Team, uno per membro). Passare a un nuovo dispositivo va bene: attivi quello nuovo, e il vecchio non può più rinnovare subito. Il suo posto si libera non appena il vecchio dispositivo si riconnette (confermando di aver ceduto l'accesso) o la sua licenza offline da 72 ore scade, quindi non c'è alcun periodo di attesa fisso. Digiti `License status` per verificare.

Cosa succede se il server delle licenze è offline?

Continua a lavorare. Dopo un'attivazione online riuscita, Buglt funziona con una licenza memorizzata in cache per un massimo di 72 ore offline, così un'interruzione (o semplicemente un weekend senza internet) non la blocca entro quella finestra. Trascorso tale periodo è necessaria una nuova verifica online.

Mi serve GitHub Copilot?

È il modo più semplice. Se non lo ha, esegua `python tools/setup_wizard.py` nel Terminale per configurarlo, e Buglt può funzionare in modalità standalone con la sua chiave OpenAI/Anthropic (`python standalone/run.py`).

I miei dati sono privati?

Sì. Taskivator riceve solo dati di licenza/aggiornamento: l'accesso al suo account Buglt (nel browser, sul Portal), un ID dispositivo con hash a senso unico e il diritto Solo o Team che approva per questo dispositivo. Il software Buglt non utilizza la misurazione di Google Ads né invia telemetria del prodotto. Non c'è alcun codice di licenza, e la sua password resta nel browser. I suoi ticket vanno solo al modello AI e al tracker che collega, mai a noi.

Posso modificare i file di Buglt?

È lei a dover personalizzare le sue parti: il glossario, lo stile dei ticket e le impostazioni (Parte 3). Solo, non disattivi la licenza/attivazione. Se trova un vero bug nello strumento stesso, scriva al supporto così può essere corretto per tutti nel prossimo aggiornamento.

Con quali bug tracker funziona?

Con undici, e Buglt registra in ognuno di essi: Jira Cloud, Azure DevOps, GitHub Issues, GitLab Issues, Bugzilla, YouTrack, Linear, Shortcut, ClickUp, Asana e Trello. Ognuno ha una configurazione guidata, ognuno scrive con una credenziale che lei crea nel proprio account, e Buglt la verifica contro la sua destinazione prima di salvarla. Ciò che cambia è quali extra arrivano: GitHub, GitLab e Bugzilla non hanno alcun caricamento di allegati, ClickUp accetta un file e non può rimuoverlo, e GitHub, GitLab e Trello non hanno un campo priorità, quindi lì la gravità viene scritta nel testo del ticket. Scegli il suo durante la configurazione e lo cambi quando vuole con `Begin setup`.

Rileverà i bug duplicati prima che io apra un ticket?

Sì, in un report standard. Prima che venga scritto qualsiasi cosa, cerca nel suo tracker report simili (anche formulati in modo un po' diverso) e le mostra le corrispondenze, così può evitare di segnalare due volte lo stesso bug. Decide comunque lei se procedere. Il modulo rapido salta quella ricerca di proposito, per risparmiare il viaggio di andata e ritorno, e l'anteprima glielo dice.

Può scrivere ticket in più di una lingua?

Sì. Aggiunga una seconda lingua durante la configurazione e ogni report viene scritto in entrambe, mantenute in blocchi separati, usando il suo glossario così i termini di prodotto restano esatti; non improvvisa mai una traduzione.

Posso allegare uno screenshot?

Chieda a Buglt di allegarli. Crea una cartella delle prove, la apre e le indica il percorso esatto; ci metta i suoi file e dica che ha finito. Buglt rimuove poi qualsiasi dato sensibile che nota, le mostra un'anteprima e li allega dopo il suo `FILE IT`. Buglt non può raggiungere un file incollato in chat, per questo le propone sempre la cartella.

Nasconde password e token nei miei report?

Quando l'oscuramento è attivo, ripulisce email, token e indirizzi IP da ogni bozza prima di aprirla. Vede sempre il testo completo nella sua anteprima per primo.

Ho aperto un ticket per errore. Posso annullarlo?

L'anteprima e la conferma digitata ([FILE IT](#)) bloccano la maggior parte degli errori prima che venga scritto qualsiasi cosa. Se ha attivato il log di controllo, digiti [Undo last filing](#) . Altrimenti, chiuda o modifichi il ticket nel suo tracker come al solito.

Può aiutarmi a verificare o chiudere un bug?

Sì. Chieda un commento di verifica e prepara una nota chiara di superato/non superato (nelle sue lingue) sul ticket esistente, e la approva lei prima che venga pubblicata.

Posso usarlo per più di un progetto?

Ogni cartella BugIt è configurata per il tracker di un solo progetto. Per un altro progetto, riesegua [Begin setup](#) per puntarlo altrove, oppure mantenga una copia separata per ogni progetto.

Come ricevo gli aggiornamenti?

Quando esce una nuova versione, BugIt glielo segnala non appena avvia una chat. Digiti [update](#) e la installa sul posto: il suo glossario, lo stile dei ticket e le impostazioni vengono conservati, e li salva prima in backup.

Cosa succede quando la mia licenza scade?

Quando il suo periodo termina, rinnovi dal suo account; il suo dispositivo applica il diritto rinnovato al successivo controllo online. La licenza offline di 72 ore descritta sopra è separata; copre solo un'interruzione temporanea del server delle licenze, non un periodo scaduto. Digiti [License status](#) in qualsiasi momento.

Se rinnovo o acquisto un'altra licenza, i giorni si sommano?

No, le licenze non si sommano. Un rinnovo **sostituisce** il suo diritto attuale; il suo periodo riparte da zero e gli eventuali giorni non utilizzati non vengono riportati, quindi rinnovi verso la fine del suo periodo.

PARTE 6 · LICENZA E ASSISTENZA

Termini di licenza (riepilogo in linguaggio semplice)

I termini completi e vincolanti si trovano nel file [LICENSE](#) nella sua cartella BugIt, e quel documento fa fede. In breve:

ARGOMENTO	IN PAROLE SEMPLICI
Concesso in licenza, non venduto	Acquista una licenza d'uso di BugIt, non la proprietà. Taskivator mantiene tutti i diritti.
Le sue postazioni	Solo = un dispositivo alla volta; Team = un massimo di 5 membri. Il suo account e il suo diritto sono personali; non li condivide, rivenda o pubblici.
Revoca	Un accesso condiviso, rimborsato, contestato o usato impropriamente può essere revocato.

Personalizza, ma...	Modifichi liberamente configurazione, glossario e template, ma non disattivi né aggiri la licenza/attivazione.
La sua responsabilità	Come usa BugIt e la sicurezza del suo progetto sono sue. Riveda e approvi ogni report prima che venga aperto. Le misure di sicurezza sono un aiuto, non una garanzia.
"Così com'è", responsabilità	Fornito così com'è, senza garanzia. Nella misura consentita dalla legge, la responsabilità è limitata a quanto pagato, escludendo danni indiretti/conseguenziali.
Durata e legge	Una licenza di un anno che inizia il giorno in cui la attiva (un acquisto una tantum, nessun rinnovo automatico), regolata dalle leggi del Giappone. I rimborsi vengono gestiti tramite lo store presso cui ha acquistato.
I rinnovi non si sommano	Un rinnovo sostituisce il suo diritto attuale: il suo periodo riparte da zero e i giorni non utilizzati non vengono riportati, quindi rinnovi verso la fine del suo periodo.

Due documenti nella sua cartella

[LICENSE](#) (termini completi) e [PRIVACY.md](#) (informativa sulla privacy). Attivando e utilizzando BugIt, accetta la LICENSE.

Ottenere assistenza

Controlli prima la Risoluzione dei problemi e le FAQ sopra; coprono quasi tutto. Poi, in quest'ordine:

1 · Chieda all'AI di risolverlo

La sua prima mossa migliore: descriva il problema in chat e chieda all'assistente di risolverlo. Se modifica un file e le sembra corretto, clicchi Keep per applicarlo (solo sul suo PC).

2 · Esegua un controllo di salute

[Check status](#) · [Check readiness](#) · oppure [python tools/selftest.py](#) nel Terminale.

3 · Ripristini

[Restore my settings](#) annulla una modifica o un aggiornamento andato male.

4 · Contatti una persona

Solo se la guida e l'AI non hanno potuto aiutarla: visiti [bugit.dev](#) o scriva a support@bugit.dev (l'assistenza è gestita in inglese). Problemi di configurazione nei primi 7 giorni? La aiuteremo a partire o con un rimborso tramite lo store.

Tenga le informazioni riservate del progetto fuori dalle email di supporto

Ogni progetto è diverso, e gran parte del suo lavoro potrebbe essere riservato. Se ci scrive, descriva il problema di BugIt solo in termini generali, e non incolli codice riservato, dettagli sui bug, specifiche, screenshot o dati del suo progetto. Taskivator non è responsabile per eventuali informazioni riservate che ci vengono inviate, e qualsiasi contenuto riservato che riceviamo viene eliminato immediatamente. Quando possibile, chieda prima aiuto all'assistente AI; può risolvere la maggior parte dei problemi direttamente nel suo editor.

Grazie per aver scelto BugIt.

Apra ticket puliti, salti i duplicati, e si riprenda il suo tempo, un bug alla volta.

BugIt QA Agent · Guida di configurazione e utilizzo · © 2026 Taskivator. All Rights Reserved. · [bugit.dev](#) · I file LICENSE e PRIVACY.md inclusi con il suo download sono i documenti di riferimento.